University of Geneva

Faculty of Science

Video Compression and Modification Detection

Digital Forensics
14x065

Michel Jean Joseph Donnet

2025-11-26

Contents

1 Introduction

Nowadays, a large number of videos circulate on the internet. Given that each video is composed of around twenty images per second, storing and transmitting videos requires a large amount of data. This is why methods have been implemented to reduce the amount of data used by a video while preserving its visual quality. However, some videos circulating on the internet convey a distorted image of reality through clever modifications to the original content, which can even lead to people being exonerated in court. This is why it is important to know the history of a video, which can be achieved through digital forensics. Since video compression leaves traces, these can be analyzed to reveal any modifications to the video. The objective of this work will therefore be to understand how video compression works and to see how the traces left by compression can be used to detect modifications to the video.

2 Methodology

In general, video are sequence of images called frames. Based on this kind of video, we will start by studying how video compression works. To do this, we will use the h264 codec, as it is the most widely used codec on the web.

2.1 Redundancies

2.1.1 Psychovisual redundancies

The imperfections of the human visual system can be exploited to reduce the amount of data used by a video without a loss of visual quality.

2.1.1.1 Spatial irrelevance

The human visual system has some difficulties to perceive small details due to its limitations. The optics of the eyes and the neural processing tend to smooth out fine patterns. For example: a document printed by a laser printer is composed of small dots that are very close together. However, we cannot see the individual dots at a normal distance: we only perceive a uniform surface.

This is why this feature can be used to remove small details that are invisible or barely visible for human visual system.

2.1.1.2 Spectral irrelevance

The human visual system consists of approximately 100 million rods responsible for perceiving brightness and approximately 6.5 million cones responsible for perceiving colors. Due to the high amount of rods, the human visual system is more sensitive to brightness compared to color.

There are 3 types of cones:
- L-cones ($\approx 65\%$), sensitive to long wavelengths (such as the red color).
- M-cones ($\approx 33\%$), sensitive to medium wavelengths (such as the green color).
- S-cones ($\approx 2\%$), sensitive to short wavelengths (such as blue color).

In this way, human visual system is less sensitive to blue color than to other colors due to its amount of S-cones.

Therefore, retaining all the spectral details of the video may prove unnecessary for good quality reproduction of the video.

2.1.1.3 Temporal irrelevance

The human visual system is not sensitive to rapid changes. Thus, in the case of a video, only about 30 frames per second are necessary for humans to perceive smooth motion. Since human visual system is unable to detect rapid changes between successive frames, this can be exploited to reduce the amount of data used by a video without loss of visual quality.

2.1.2 Statistical redundancies

The statistical redundancies of each frame of a video can be used to optimize the amount of data used. Among these redundancies, we have spatial, spectral, and temporal redundancies, as in the human visual system.

2.1.2.1 Spatial irrelevance

These redundancies are introduced by a strong correlation between neighboring pixels. By the way, in an image, each pixel is correlated with its neighbors in such a way that the final result is something visible and understandable to humans. An image created with no correlation between pixels is something like a noisy image because each pixel is independent from the others. So the spatial redundancies can be exploited to predict for instance values of pixels according to neighboring pixels. In this manner, some data can be deleted, which reduces amount of data used.

2.1.2.2 Spectral irrelevance

These redundancies are created by strong correlation between neighboring pixels in color domain. This is because color changes are often gradual, and colored regions regularly extend beyond a single pixel. Thus, a pixel is strongly correlated with its neighbors in the color domain, which can be exploited to reduce amount of data used.

2.1.2.3 Temporal irrelevance

These redundancies are the most important for video compression. In fact, changes to the elements present in the video occur gradually, especially in consecutive frames. Thus, between two consecutive frames, there will not be many changes, as shown by Figure 1. Indeed, the amount of change created by this moving car is very small.

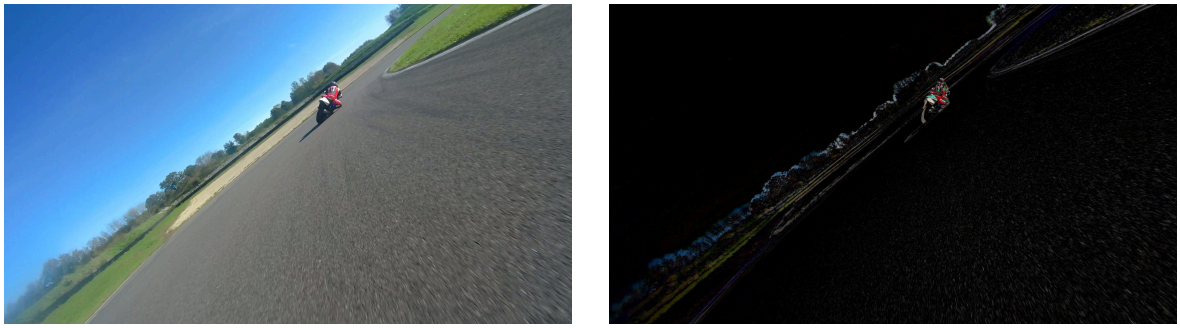

Figure 1: Difference between 2 consecutive frames for a 60 fps video

So if only the changes are recorded, most of the frame can be compressed.

2.2 H.264

The H.264 codec, also known as MPEG-4 AVC (Advanced Video Coding) or MPEG-4 Part 10, was developed in 2003. It undergoes numerous transformations and is still undergoing improvements today [1].

2.2.1 Compression

Video compression follows steps similar to those used in image compression, which consist of data transformation, quantization for lossy compression, and data encoding for efficient

[1] Wikipedia

storage on disk. However, some additional steps are provided to exploit temporal redundancies.

More concretely, we have the steps bellow.

2.2.1.1 Color Space Transform

The frame is first transformed from RGB color space to YCbCr color space. Instead of representing the frame in RGB color space, the YCbCr color space is used to divide the frame into luminance (Y) and chrominance (both Cb and Cr).

Y channel Cb channel Cr channel

Figure 2: Image decomposed in YCbCr color space

The luminance looks after the luminosity, the brightness of the image whereas the chrominance looks after colors of the image, as shown on Figure 2. On top of that, as we can observe on Figure 2, all structural information is contained in Y channel, which represent clearly the image in grayscale.

In this way, since the human visual system is less sensitive to color than to brightness, chrominance could be compressed more than luminance at a later stage.

2.2.1.2 Split into macro-blocks

Then, the frame is divided into macro-block generally of size 16×16. These macro-blocks perfectly represent specific regions of the image and are very useful for working on small areas of the image. This allows for easier detection of elements such as moving and not moving objects, more accurate motion prediction, and an efficient means of compressing data. Indeed, some parts of the image may be flat while others may contain a large amount of detail. And on subsequent frames, some informations can be exactly the same, as represented by the black on Figure 1. In this way, some parts of the image can be compressed more than others.

2.2.1.3 Partitioning

Each macro-block is then divided into smaller blocks depending on precision needed. This allows a better adaptation on local complexity of the frame.

The macro-block can be divided into sub-blocks of size 16×16, 8×8, 8×4, 4×8 or 4×4.

2.2.1.4 Prediction

This step is divided into two sub-steps.

2.2.1.4.1 Intra-prediction

In this case, only the current frame is taken into account. The goal is to predict the value of a pixel based on its neighborhood. The prediction mode is chosen by the encoder, such as vertical, horizontal or other modes.

- vertical mode: copy content of previous row
- horizontal mode: copy content of previous column

The prediction that gives the smallest difference between original block frame and predicted block frame is kept according to chosen mode. There exists 9 prediction modes for 4×4 blocks and 4 modes for 16×16 blocks.

The intra-prediction is very useful for scenes without moves.

2.2.1.4.2 Inter-prediction

In this case, the subsequent frames are used to make the prediction (past and future frames, depending on kind of prediction). The goal is to predict block position according to subsequent frames. This is based on the fact that only small changes are introduced between subsequent frames, as shown on Figure 1.

First of all, we need to introduce a few technical terms, summarized in the Table 1.

I-frame This is an independent frame that is fully encoded using Section 2.2.1.4.1, which results in low prediction efficiency. The file size is large due to the amount of data that needs to be retained. This type of frame corresponds to the key frames in the video, which are used to encode the other frames.

P-frame This is a frame encoded using the previous I-frame, which results in good prediction efficiency thanks to previous I-frame. The aim is to track the movement of the object in the video and use this to reduce the amount of data used. By exploiting this source of redundancy, the amount of data used is less than that of I-frame, so the compression is better.

B-frame this frame is encoded using the previous I-frame and the following P-frame, which results in high prediction efficiency. By using the previous and following images, the prediction is definitely better, allowing less data to be used than for P-frames. This kind of frame is especially used for compression.

Frame type	Dependency	Prediction efficiency	File size	Goal
I-frame	No	Bad	-	Key frame
P-frame	I-frame	Good	+	Track move
B-frame	I-frame, P-frame	High	+++	Compression

Table 1: Kind of frames used in a video

There is no inter-prediction in I-frame, as described in Table 1.

On case of P-frame computation, the block of the P-frame is searched in the previous I-frame. A motion vector is then computed, describing the move of the block between this two frames. Then, the content of the reference block is predicted according to motion vector and I-frame content.

On case of B-frame computation, the block of the B-frame is searched in the previous I-frame and next P-frame. The motion vector is then computed, allowing prediction of the block according to I-frame and P-frame. In this case, the prediction of the block is more accurate thanks to the I-frame and P-frame used, allowing greater data compression later on.

Computing the motion vector is costly, however it allows to achieve great compression on moving objects for P-frames and B-frames.

The structure of the video follows a pattern as shown in the Figure 6.

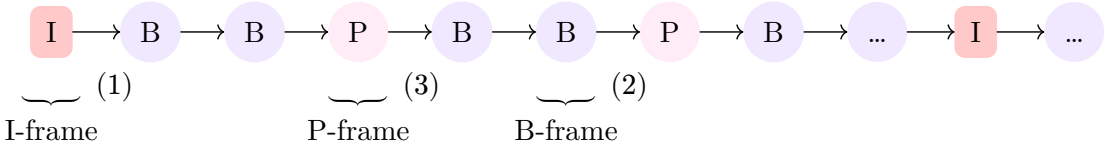

Figure 6: Frame organization in video

2.2.1.5 Residue

Once the prediction is made for a block, the difference between the predicted block and the original block of the frame is computed. Depending on results, prediction can be reapplied to have smaller residuals allowing better compression, as mentioned in Section 2.2.1.4.1.

In this way, only the method for establishing the prediction and the residual difference between the prediction and the original image are necessary to reconstruct the original image.

For instance, suppose that we have B_{original} the original block of the frame and $B_{\text{predicted}}$ the predicted block. Then, the residual R can be computed as shown in Equation 4.

$$R = B_{\text{original}} - B_{\text{predicted}} \tag{4}$$

So if only both motion vector for inter-prediction and prediction method for intra-prediction is kept with the residual, the original block can be reconstruct as shown in Equation 5.

$$B_{\text{original}} = R + B_{\text{predicted}} \tag{5}$$

This is why only residual of prediction and information necessary for prediction are retained, allowing for considerable data compression.

2.2.1.6 Transform

An integer approximation of the Discrete Cosine Transform (DCT) is applied to the prediction residual in order to convert it into frequency coefficients.

The integer approximation is used to facilitate computations and avoid rounding error introduced by a classical DCT that works with floating points. In fact, the entire approximation of the DCT is reversible and only processes integers, which is particularly relevant when working with images and videos in the integer domain.

In this way, the residual is divided into high and low frequencies, which proves useful in the next step: the quantization.

2.2.1.7 Quantization

The quantization is a lossy step that determine the quality of the compressed video. It is based on imperfection of human visual system, which is less sensitive to small details, as explained in Section 2.1.1. Since high frequencies represent small details, quantization uses the decomposition of the residue into low and high frequencies to remove the high frequencies. To do this, a quantization matrix is constructed based on the quality factor defined by the user. Then, the residual is divided by the quantization matrix and the result is rounded to

have only integer values. Some high frequencies then become 0, which reduces the amount of data to be stored. In this way, some data are lost, that's why this is a lossy step.

2.2.1.8 Encoding

Finally, the quantified residues and parameters useful for prediction, such as the motion vector or prediction method, are encoded in such a way as to use as few bits as possible and to be able to reconstruct the encoded video.

> **Note**
>
> A very good explanation of how H264 works is available on the website abhik.xyz, which provides interactive explanations.

2.2.2 Decompression

The process of decompression follows the same steps as video compression, but in reverse order, as shown on Figure 7.

So first, data are decoded. Next, as in the quantization step for encoding, the DCT coefficients were divided by a quantization matrix, the DCT coefficients are multiplied by the quantization matrix to recover the DCT coefficients in the inverse quantization step. Then, an inverse DCT is applied to retrieve the residues, which are added to the prediction made. In this way, the original blocks of the partitioning are retrieved, as explained in Section 2.2.1.5. Then, blocks are reassembled to reconstruct the video. Finally, deblocking filters are applied to avoid blocking artifacts on the video.

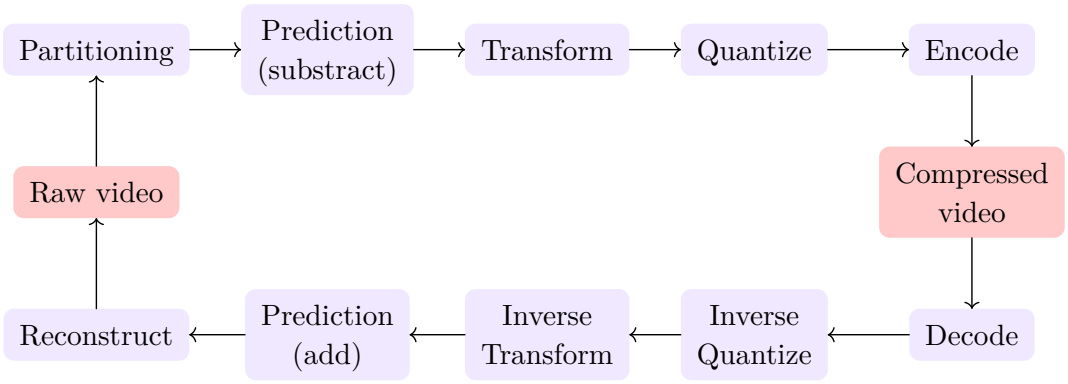

Figure 7: General scheme of video compression/decompression

2.2.3 H265

The H265 codec, also known as High Efficiency Video Coding (HEVC), has emerged in 2013 and is still in development. It is based on h264 and improves upon it in terms of compression quality. The H265 codec therefore follows the compression scheme described in Figure 7, with a few modifications that make it more efficient in terms of compression, as explained below.

2.2.3.1 Compression

The H265 compression follows the same steps than the H264 compression. However, both H265 encoding and decoding is made to work in parallel, allowing less power consumption. But this requires better material, making it not fully compatible with all devices.

2.2.3.1.1 Partitioning

This is the main step that makes H265 more efficient than H264. Instead of working with a decomposition into macro-blocks, the frame is decomposed in Coding Tree Unit (CTU). The Coding Tree Unit divides each frame in Coding Tree (CT) of size 64×64. Next, each coding tree is divided into sub-blocks called coding units (CUs), ranging in size from 64×64 to 8×8, depending on requirements. The figure blabla show an example of decomposition of a Coding Tree into Coding Units. So thanks to the large size that Coding Units can take, the compression of flat areas, like for instance a blue sky, is better than for H264.

2.2.3.1.2 Prediction

The prediction is made on Coding Units. During the prediction step, the Prediction Unit (PU) is created. It contains all information necessary to perform the prediction, such as prediction mode used or sub-decomposition of the Coding Unit.

Intra-prediction support handles 35 different modes, compared to 9 for H264, enabling more accurate prediction despite higher computational complexity. This way, the prediction is more accurate, creating a smaller prediction error that can be more compressed.

Inter-prediction is an enhancement of the inter-prediction of H264. A motion vector prediction refinement is calculated, enabling better motion tracking. On top of that, the search of the move of a block can be performed on blocks with different sizes, against H264 where the search was only between blocks of size 16×16.

So the prediction error contains less informations, which allows a better compression.

3 Implementation

4 Results

5 Discussion

6 Conclusion

Bibliography